

Обработчики, задаваемые на методы бизнес-логики

Обработчик — это особая разновидность метода, который автоматически выполняется при вызове метода БЛ, которому он принадлежит. С помощью обработчиков можно расширить базовый алгоритм автогенерируемых, списочных или файловых методов бизнес-логики, и, как следствие, оптимизировать приложение, потому как применение обработчиков:

- уменьшает дублирование исходного кода методов БЛ;
- уменьшает вероятность допустить ошибки, заново реализуя базовый алгоритм;
- исходный код методов БЛ становится более поддерживаемым.

Реализовать обработчик можно на языках программирования Python или C++, выбор которого зависит от предпочтений разработчика и не связан с реализацией метода БЛ, к которому создан обработчик. Обработчики отличаются от методов БЛ тем, что их невозможно явно вызвать из клиентской части приложения.

Создание обработчика производят в [Genie](#) в диалоге редактирования метода БЛ. Диалог с настройками обработчика вынесен в отдельную вкладку Handler. Список опций отличается для каждого типа метода, что отражено в соответствующих разделах:

- [Автогенерируемые методы](#)
- [Списочный метод](#)
- [Файловый метод](#)

Автогенерируемые методы

Ниже представлен список параметров, используемых для конфигурации обработчиков:

Аргументы	Описание
Name	Имя обработчика. Должно быть уникальным в рамках автогенерируемого метода. Для обработчиков с типом реализации Native function (см. Implementation type) имя совпадает с первым аргументом макроса HANDLER или HDL (см. Встроенная функция). На имя такого обработчика накладывается ограничение: оно должно быть задано по-английски. Для обработчиков с одинаковым типом (см. Handler type) по имени определяется лексикографический порядок их выполнения.
Implementation type	Тип реализации метода. • Python. Исходный код обработчика описывается в Genie в диалоге создания обработчика. • Native function (реализация на C++). Исходный код обработчика описывается в отдельном DLL-файле. Правила создания и оформления файла, а также особенности объявления обработчика описаны в разделе Как создать обработчик с типом реализации Native function (C++).
Order type	Устанавливает порядок выполнения обработчика относительно тех, что унаследованы от родительского объекта БЛ. Возможные значения: • Execute before parent's methods. Выполнять перед всеми обработчиками, которые унаследованы от родительского объекта БЛ.

	- Execute after parent's methods. Выполнять после всех обработчиков, которые унаследованы от родительского объекта БЛ. - Replace parent's methods. Все обработчики, унаследованные от родительского объекта БЛ, выполняться не будут, а вместо них будет вызван данный обработчик.
Handler type	Тип обработчика. Определяет момент выполнения обработчика: до или после вызова основного метода.

Обработчик Initialization

Обработчик автогенерируемого метода [Create](#).

Аргументы	Описание
Запись	Объект, который сформирован в результате работы автогенерируемого метода Create. В обработчике вы можете изменить значения полей этого объекта, тогда инициализируемый объект также будет изменён.
Фильтр	Объект, который передан при вызове автогенерируемого метода Create в одноимённый аргумент "Фильтр".
ИмяМетода	Значение одноимённого аргумента, который передан при вызове автогенерируемого метода Create.
[deprecated] Параметры	Аргумент "Параметры" признан устаревшим и не должен использоваться вами при разработке прикладных решений. По умолчанию в аргумент приходит пустой объект. Аргумент оставлен для поддержания совместимости с устаревшим функционалом. В прошлых версиях Genie объектам БЛ можно было задавать параметры, значения которых использовали обработчики методов. Функционал не нашёл широкого применения, поэтому в текущей реализации от параметров отказались. Чтобы поддержать работу обработчиков, использующих такие параметры, в аргумент "Параметры" передаётся объект с соответствующими параметрами объекта БЛ.

 Здесь "объект" — это экземпляр класса [sbis.Record](#) (для Python) или [sbis::Record](#) (для C++).

Обработчик Before Create

Обработчик выполняется перед вызовом автогенерируемых методов [Create](#) или [Update](#). Подробнее об этом читайте [здесь](#).

Аргументы	Описание
Запись	Объект, который сформирован в результате работы автогенерируемого метода Create. В обработчике вы можете изменить поля этого объекта, тогда и создаваемый черновик или объект БД тоже будет изменён.
Фильтр	Объект, который передан при вызове автогенерируемого метода Create в одноимённый аргумент "Фильтр".
[deprecated] Параметры	Аргумент "Параметры" признан устаревшим и не должен использоваться вами при разработке прикладных

решений. По умолчанию в аргумент приходит пустой объект*.

Аргумент оставлен для поддержания совместимости с устаревшим функционалом. В прошлых версиях Genie объектам БЛ можно было задавать параметры, значения которых использовали обработчики методов. Функционал не нашёл широкого применения, поэтому в текущей реализации от параметров отказались. Чтобы поддержать работу обработчиков, использующих такие параметры, в аргумент "Параметры" передаётся объект с соответствующими параметрами объекта БЛ.

Обработчик After Create

Обработчик выполняется либо после вызова автогенерируемого метода [Create](#), либо перед вызовом [Update](#). Подробнее об этом читайте в разделе [Особенности вызова обработчиков для автогенерируемых методов Create и Update](#).

Аргументы	Описание
Запись	Объект, который сформирован в результате работы автогенерируемого метода Create. В момент выполнения обработчика объект уже добавлен в БД в качестве черновика .
Фильтр	Объект, который передан при вызове автогенерируемого метода Create в одноимённый аргумент "Фильтр".
[deprecated] Параметры	Аргумент "Параметры" признан устаревшим и не должен использоваться вами при разработке прикладных решений. По умолчанию в аргумент приходит пустой объект. Аргумент оставлен для поддержания совместимости с устаревшим функционалом. В прошлых версиях Genie объектам БЛ можно было задавать параметры, значения которых использовали обработчики методов. Функционал не нашёл широкого применения, поэтому в текущей реализации от параметров отказались. Чтобы поддержать работу обработчиков, использующих такие параметры, в аргумент "Параметры" передаётся объект с соответствующими параметрами объекта БЛ.

Обработчик Before Read

Обработчик автогенерируемого метода [Read](#).

Аргументы	Описание
ИдО	Значение одноимённого аргумента "ИдО", который передан при вызове автогенерируемого метода Read.
ИмяМетода	Значение одноимённого аргумента "ИмяМетода", который передан при вызове автогенерируемого метода Read.
ИмяСвязи	Значение одноимённого аргумента "ИмяСвязи" (см. Новое поведение метода Read), который передан при вызове автогенерируемого метода.

Обработчик After Read

Обработчик автогенерируемого метода [Read](#). Изменение аргументов обработчика никак не влияет на результат выполнения автогенерируемого метода.

Аргументы	Описание

ИдО	Значение одноимённого аргумента "ИдО", который передан при вызове автогенерируемого метода Read.
ИмяМетода	Значение одноимённого аргумента "ИмяМетода", который передан при вызове автогенерируемого метода Read.
ИмяСвязи	Значение одноимённого аргумента "ИмяСвязи" (см. Новое поведение метода Read), который передан при вызове автогенерируемого метода.
Запись	Объект , полученный в результате выполнения автогенерируемого метода Read.

Обработчик Before Update

[Обработчик](#) автогенерируемого метода [Update](#).

Аргументы	Описание
Запись	Объект, который получен в результате изменений, выполненных предыдущим обработчиком (из цепочки обработчиков) с типом "Before Update". Если данный обработчик является первым в "цепочке обработчиков", то значением аргумента является объект, созданный объединением объектов из аргументов "Старая Запись" и "Фильтр". Порядок выполнения обработчика задаётся в параметре Order type (см. Параметры обработчиков автогенерируемого метода).
Фильтр	Объект, который передан при вызове автогенерируемого метода Update в одноимённом аргументе "Запись".
[deprecated] Параметры	Аргумент "Параметры" признан устаревшим и не должен использоваться вами при разработке прикладных решений. По умолчанию в аргумент приходит пустой объект. Аргумент оставлен для поддержания совместимости с устаревшим функционалом. В прошлых версиях Genie объектам БЛ можно было задавать параметры, значения которых использовали обработчики методов. Функционал не нашёл широкого применения, поэтому в текущей реализации от параметров отказались. Чтобы поддержать работу обработчиков, использующих такие параметры, в аргумент "Параметры" передаётся объект с соответствующими параметрами объекта БЛ.
Старая запись	Объект (запись БД), который хранится в БД и будет обновлён.

Обработчик After Update

[Обработчик](#) автогенерируемого метода [Update](#). Изменение аргументов обработчика никак не влияет на результат выполнения автогенерируемого метода.

Аргументы	Описание
Запись	Объект, который получен в результате изменений, выполненных предыдущим обработчиком (из цепочки обработчиков) с типом "After Update". Если данный обработчик является первым в "цепочке обработчиков", то значением аргумента является объект, созданный объединением объектов из аргументов "Старая Запись" и "Фильтр". Порядок выполнения обработчика задаётся в параметре Order type (см. Параметры обработчиков автогенерируемого метода).

Фильтр	Объект, который передан при вызове автогенерируемого метода Update в одноимённом аргументе "Запись".
[deprecated] Параметры	Аргумент "Параметры" признан устаревшим и не должен использоваться вами при разработке прикладных решений. По умолчанию в аргумент приходит пустой объект. Аргумент оставлен для поддержания совместимости с устаревшим функционалом. В прошлых версиях Genie объектам БЛ можно было задавать параметры, значения которых использовали обработчики методов. Функционал не нашёл широкого применения, поэтому в текущей реализации от параметров отказались. Чтобы поддержать работу обработчиков, использующих такие параметры, в аргумент "Параметры" передаётся объект с соответствующими параметрами объекта БЛ.
Старая запись	Объект, который хранился в БД и был перезаписан.

Обработчик Before Copy

Обработчик автогенерируемого метода [Copy](#).

Аргументы	Описание
Старый Внимание: аргумент назван правильно, но хранимое в нём значение логически не соответствует подразумеваемому.	Копия прочитанного объекта.
Новый Внимание: аргумент назван правильно, но хранимое в нём значение логически не соответствует подразумеваемому.	Объект, который прочитан автогенерируемым методом Copy по параметру "ИдО".

Обработчик After Copy

Обработчик автогенерируемого метода [Copy](#).

Аргументы	Описание
Старый Внимание: аргумент назван правильно, но хранимое в нём значение логически не соответствует подразумеваемому.	Копия прочитанного объекта.
Новый Внимание: аргумент назван правильно, но хранимое в нём значение логически не соответствует подразумеваемому.	Объект, который прочитан автогенерируемым методом Copy по параметру "ИдО".

Обработчик Before Delete

Обработчик автогенерируемого метода [Delete](#). Изменение аргументов обработчика никак не влияет на результат выполнения автогенерируемого метода.

Аргументы	Описание
Запись	Объект, который будет удалён из БД.

Обработчик Before Batch Delete

Обработчик автогенерируемого метода [Delete](#). Изменение аргументов обработчика никак не влияет на результат выполнения автогенерируемого метода.

Аргументы	Описание
Записи	Объекты, которые будут удалены из БД.

Здесь "объекты" — это экземпляр класса [sbis.RecordSet](#) (для Python) или [sbis::RecordSet](#) (для C++).

Обработчик After Delete

Обработчик автогенерируемого метода [Delete](#). Изменение аргументов обработчика никак не влияет на результат выполнения автогенерируемого метода.

Аргументы	Описание
Запись	Объект, который был удалён из БД.

Обработчик After Batch Delete

Обработчик автогенерируемого метода [Delete](#). Изменение аргументов обработчика никак не влияет на результат выполнения автогенерируемого метода.

Аргументы	Описание
Записи	Объекты, которые будут удалены из БД.

Обработчик Before Merge

Обработчик автогенерируемого метода [Merge](#). Изменение аргументов обработчика никак не влияет на результат выполнения автогенерируемого метода.

Аргументы	Описание
Запись	Объект, с которым происходит объединение.
Удаляемый	Объект, который будет удалён после объединения.

Обработчик After Merge

Обработчик автогенерируемого метода [Merge](#). Изменение аргументов обработчика никак не влияет на результат выполнения автогенерируемого метода.

Аргументы	Описание
Запись	Объект, полученный в результате объединения.
Удаляемый	Объект, который был удалён после объединения.

Особенности вызова обработчиков для автогенерируемых методов Create и Update

Порядок выполнения и набор обработчиков для автогенерируемых методов Create и Update связан с флагом "Insert record into a table on execution of auto-generated Create method", который задаётся в настройках родительского объекта БЛ.

Если флаг установлен, то обработчики метода Create выполняются в следующей последовательности: обработчик Initialization → обработчик Before Create → метод Create → обработчик After Create.

В результате будет создан т.н. [черновик](#), который ещё не является полноценной записью.

Для вставки записи в БД требуется вызвать автогенерируемый метод Update, что может быть инициировано, например, из пользовательского интерфейса.

Обработчики метода Update выполняются в следующей последовательности: обработчик Before Update → метод Update → обработчик After Update.

Если флаг не установлен, то при вызове метода Create выполняется только обработчик Initialization.

Далее при первом вызове автогенерируемого метода Update выполняются следующие обработчики: обработчик Before Create → метод Create (на базе выполняется INSERT) → обработчик After Create.

При последующих вызовах Update: обработчик Before Update → метод Update (на базе выполняется UPDATE) → обработчик After Update.

Обработчики методов Create и Update — это, скорее, триггеры INSERT и UPDATE на уровне бизнес-логики, поэтому им присуще такое поведение.

Что должен возвращать обработчик автогенерируемого метода?

Рассмотрим интерпретацию результатов, которые возвращает обработчик:

- `true` — обработчик выполнен без ошибок, продолжается выполнение следующего программного кода.
- `false` — означает, что в результате выполнения алгоритма обработчика получена какая-то ошибка.

В результате происходит откат транзакции, в рамках которой выполнялись изменения. Также в исходном коде создаётся исключение, а в пользовательском интерфейсе выводится сообщение об ошибке вида "Ошибка в результате выполнения обработчика автогенерируемого метода".

В details ошибки не будет записана информативная причина ошибки, а только `false`. Поэтому вместо `false` рекомендуется самостоятельно инициировать создание исключения из исходного кода обработчика, чтобы подробно описать причину ошибки.

Таким образом, работа с приложением станет "дружелюбной".

- `none` — равнозначен `true`.

Списочный метод

Обработчики списочных методов на Python позволяют перенести логику расчетных полей с сервера БД на сервер бизнес-логики, что позволяет масштабировать нагрузку на СУБД, упростить и оптимизировать непосредственно сами SQL запросы, снизив вычислительные операции пост-обработки полученных наборов записей.

Пример:

```
1 @staticmethod
2 def List_AfterList(params, result):
3     i = 0
4     while i < result.Size():
5         if result[i]['НашаОрганизация.@Лицо'].IsNull():
6             result.DelRow(i)
7         else:
8             i += 1
9     return True
```

- `params` — [MethodListParams](#) — входные параметры списочного метода,
- `result` — [MethodListResult](#) — результат выполнения списочного метода.

Обработчик списочного метода через in-out аргумент `result` должен возвращать список — экземпляр класса [sbis.RecordSet](#) (для Python) или [sbis::RecordSet](#) (для C++).



Порядок выполнения обработчика

Выполняется после основного метода.

Создание обработчика на C++ выполняется согласно [общим правилам](#) с фиксированным макросом.

Как создать обработчик с типом реализации Native function (C++)

Правила объявления обработчика

1. Название обработчика должно быть уникальным. В именовании используйте английские буквы.
2. Обработчик должен возвращать true или false. Если обработчик возвращает false, то все остальные обработчики уже не будут выполнены. В методе, для которого вызван обработчик, происходит откат всех изменений в рамках транзакции, и, соответственно, откат самой транзакции.

Список общих аргументов для любых обработчиков

Следующие аргументы всегда передаются последними:

- context — это дополнительно передаваемая информация (параметры) через http-запрос.
- params — набор параметров объекта БЛ. Аргумент признан устаревшим и не должен использоваться вами при разработке прикладных решений. По умолчанию в аргумент приходит пустой объект (экземпляр класса Record).

Аргумент оставлен для поддержания совместимости с устаревшим функционалом. В прошлых версиях Genie объектам БЛ можно было задавать параметры, значения которых использовали обработчики методов. Функционал не нашёл широкого применения, поэтому в текущей реализации от параметров отказались. Чтобы поддержать работу обработчиков, использующих такие параметры, в аргумент "Параметры" передаётся объект с соответствующими параметрами объекта БЛ.

Пример:

```
1  INITIALIZATION_HANDLER( L"OnInitialization", MyFunctionInit )
2  ( Record& rec, const Record& filter, const std::string& name, IContext&
   context, const Record& params )
3  {
4      // ...
5      return true;
6  }
```

Обработчик для автогенерируемого метода

Синтаксис объявления обработчика

```
1  *_HANDLER( "Name", НазваниеФункцииНаC++ )( Сигнатура )
2  {
3      // реализация функции
4      return true;
5  }
```

Здесь *_HANDLER является обозначением макроса. Допускаются к использованию следующие макросы:

- INITIALIZATION_HANDLER
- BEFORE_CREATE_HANDLER
- AFTER_CREATE_HANDLER
- BEFORE_UPDATE_HANDLER
- AFTER_UPDATE_HANDLER
- BEFORE_READ_HANDLER
- AFTER_READ_HANDLER
- BEFORE_COPY_HANDLER
- AFTER_COPY_HANDLER
- BEFORE_DELETE_HANDLER
- AFTER_DELETE_HANDLER
- BEFORE_MERGE_HANDLER
- AFTER_MERGE_HANDLER

Также допускается использование макросов-события вида *_HDL, для которых не описывают функцию C++.

Список аргументов в сигнатуре каждого метода состоит из параметров самого обработчика и общих параметров.


```

1 *_HDL( "Name" )( Сигнатура метода )
2 {
3     // реализация функции
4     return true;
5 }

```

Примеры описания встроенных функций в качестве обработчиков автогенерируемых методов

- Initialization

```

1 INITIALIZATION_HANDLER( L"OnInitialization", MyFunctionInit )
2 ( Record& rec, const Record& filter, const std::string& name,
3   IContext& context, const Record& params )
4 {
5     return true;
6 }

```

- Before Create

```

1 BEFORE_CREATE_HANDLER( L"OnBeforeCreate", MyFunctionBeforeCreate )
2 ( Record& rec, const Record&, IContext&, const Record& )
3 {
4     return true;
5 }

```

- After Create

```

1 AFTER_CREATE_HANDLER( L"OnAfterCreate", MyFunctionAfterCreate )
2 ( const Record& rec, const Record&, IContext&, const Record& )
3 {
4     return true;
5 }

```

- Before Update

```

1 BEFORE_UPDATE_HANDLER( L"OnBeforeUpdate", MyFunctionBeforeUpdate )
2 ( Record& rec, const Record& rec_write, IContext& ctx, const Record&,
3   const Record& old_rec )
4 {
5     return true;
6 }

```

- After Update

```

1 AFTER_UPDATE_HANDLER( L"OnAfterUpdate", MyFunctionAfterUpdate )
2 ( const Record& rec, const Record& rec_write, IContext& ctx, const
3   Record&, const Record& old_rec )
4 {
5     return true;
6 }

```

- Before Read

```

1 BEFORE_READ_HANDLER( L"OnBeforeRead", MyFunctionBeforeRead )
2 ( Int64, sbis::String const&, sbis::String const&, IContext& )
3 {
4     return true;
5 }

```

- After Read

```

1 AFTER_READ_HANDLER( L"OnAfterRead", MyFunctionAfterRead )
2 ( Int64 id, String const& method_name, String const& rel_name,
3   Record& rec, IContext& )
4 {

```

```

4     return true;
5 }

```

- Before Copy

```

1 BEFORE_COPY_HANDLER( L"OnBeforeCopy", MyFunctionBeforeCopy )
2 ( const Record&, const Record&, IContext& )
3 {
4     return true;
5 }

```

- After Copy

```

1 AFTER_COPY_HANDLER( L"OnAfterCopy", MyFunctionAfterCopy )
2 ( const Record&, const Record&, IContext& )
3 {
4     return true;
5 }

```

- Before Delete

```

1 BEFORE_DELETE_HANDLER( L"OnBeforeDelete", MyFunctionBeforeDelete )
2 ( Record const& rec, IContext& )
3 {
4     return true;
5 }

```

- After Delete

```

1 AFTER_DELETE_HANDLER( L"OnAfterDelete", MyFunctionAfterDelete )
2 ( Record const& rec, IContext& )
3 {
4     return true;
5 }

```

- Before Merge

```

1 BEFORE_MERGE_HANDLER( L"OnBeforeMerge", MyFunctionBeforeMerge )
2 ( const Record&, const Record&, IContext& )
3 {
4     return true;
5 }

```

- After Merge

```

1 AFTER_MERGE_HANDLER( L"OnAfterMerge", MyFunctionAfterMerge )
2 ( const Record&, const Record&, IContext& )
3 {
4     return true;
5 }

```

Обработчик для списочного метода

Для описания обработчиков списочных методов, реализованных на C++, используется макрос `METHOD_LIST_HANDLER`.

В качестве аргументов обработчику передаются:

- `ctx` — ссылка на контекст.
- `params` — ссылка на входные параметры метода "Список" (экземпляр класса [MethodListParams](#)).
- `result` — ссылка на результат (экземпляр класса [MethodListResult](#)).

Пример:

```

1 METHOD_LIST_HANDLER( "MyDeclarativeHandler", DeclarativeSelectHandler )
2 ( IContext& ctx, MethodListParams& params, MethodListResult& result )
3 {
4     // ...

```

```

4     return true;
5 }

```

Обработчик для методов чтения файлов

Для реализации обработчиков на C++ нужно воспользоваться макросом `HANDLER_METHOD` (макрос описан в файле "sbis-bl-service/methodhandlerlist.h"), его синтаксис выглядит следующим образом:

```

1 HANDLER_METHOD( L"MyFileHandler", Название Функции на C++ )( Сигнатура
  метода )
2 {
3     // Реализация функции
4 }

```

Пример. Определение обработчика для метода "ПрочитатьФайл":

```

1 HANDLER_METHOD( L"OnReadFile", OnReadFile )( RpcFile& rpc_file,
  IContext& context )
2 {
3     //здесь мы подменяем текущее имя файла на новое имя
4     rpc_file.SetName("NewName");
5     return true;
6 }

```

Механизм перекрытия обработчиков по имени

Для одноименных обработчиков одного метода бизнес-логики, описанных в разных модулях, реализован механизм перекрытия по имени.

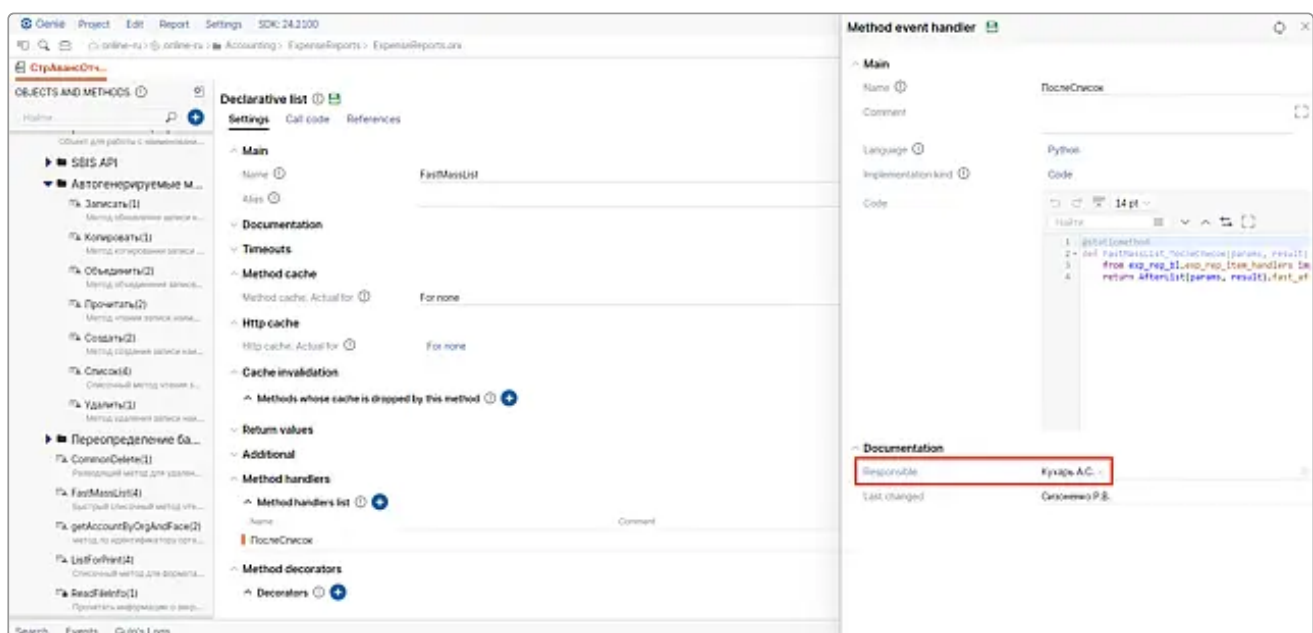
Механизм работает при одновременном выполнении условий:

- в Модуль1 описан обработчик метода и в Модуль2 описан одноименный обработчик того же метода бизнес-логики, с тем же числом аргументов,
- Модуль2 по зависимостям загружается после Модуль1.

В результате будет использоваться реализация из Модуль2.

Ответственный за обработчик

В настройках обработчика (раздел **Method handlers list**) можно указать или менять ответственного (поле **Responsible**).



При создании нового обработчика ответственный указывается автоматически.

